

SIMATS ENGINEERING ASSESSMENT TOOLS

Course Name: Theory of Computation **Course Code:** CSA1304

Course Outcome (CO5): Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)

Assessment Tool: Mock Interview

Question 1

What is a Pushdown Automaton (PDA)? How is it different from a Finite Automaton?

Answer:

A Pushdown Automaton (PDA) is a computational model that extends a Finite Automaton by adding an auxiliary stack (a last-in-first-out memory store). It is formally defined as a 7-tuple and is used to recognize Context-Free Languages (CFLs).

Difference from a Finite Automaton: an FA has no memory beyond its finite set of states, so it can only recognize Regular Languages and cannot count or match nested structures (e.g., $a^n b^n$). A PDA has an unbounded stack in addition to states, which lets it remember an arbitrary amount of information (such as a count) and therefore recognize a strictly larger class of languages — Context-Free Languages — that properly contains the Regular Languages.

Question 2

Define the components of a PDA formally.

Answer:

A PDA is formally defined as a 7-tuple: $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, where:

- Q — a finite set of states.
- Σ — the input alphabet (finite set of input symbols).
- Γ — the stack alphabet (finite set of stack symbols).
- δ — the transition function: $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{finite subsets of } Q \times \Gamma^*$.
- q_0 — the initial/start state, $q_0 \in Q$.
- Z_0 — the initial stack symbol (bottom-of-stack marker), $Z_0 \in \Gamma$.
- F — the set of final/accepting states, $F \subseteq Q$.

Question 3

What is a stack in PDA? Why is it important?

Answer:

The stack is an auxiliary, unbounded last-in-first-out (LIFO) memory structure attached to the PDA. At each step, the PDA can push a symbol onto the stack, pop the top symbol off it, or leave it unchanged, and its decisions can depend on the symbol currently at the top of the stack.

It is important because it is the only source of unbounded memory the PDA has. Without it, the machine would be a plain finite automaton limited to Regular Languages. The stack allows the PDA to 'remember' an arbitrary count (e.g., the number of a's seen) and later verify a matching condition (e.g., the same number of b's), which is exactly what is needed to recognize nested/matching structures typical of Context-Free Languages.

Question 4

What is an Instantaneous Description (ID) in PDA?

Answer:

An Instantaneous Description (ID) is a snapshot of the complete configuration of a PDA at a given moment during its computation. It is usually written as a triple (q, w, γ) , where q is the current state, w is the remaining unread portion of the input string, and γ is the current contents of the stack (written top-to-bottom or bottom-to-top by convention).

IDs are used to formally trace and describe how a PDA moves from one configuration to another as it consumes input and manipulates the stack, and the relation ' $\vdash$ ' (derives/yields in one move) is used to connect successive IDs.

Question 5

What are the different types of moves in a PDA?

Answer:

- Input-consuming move: the PDA reads one input symbol, changes state based on the current state, input symbol, and stack top, and updates the stack (push/pop/no change).
- Epsilon (ϵ) move: the PDA changes state and/or manipulates the stack without consuming any input symbol; it depends only on the current state and stack top.
- Push move: a symbol (or string of symbols) is placed on top of the stack, growing it.
- Pop move: the top symbol of the stack is removed.

- No-change move: the stack top symbol is replaced by itself or left untouched depending on the transition definition.

Question 6

What is acceptance by final state and empty stack?

Answer:

Acceptance by final state: the PDA accepts an input string if, after reading the entire string, it is in one of the designated final/accepting states (regardless of what remains on the stack).

Acceptance by empty stack: the PDA accepts an input string if, after reading the entire string, its stack becomes completely empty (regardless of the current state, and often the set of final states F is simply left empty in this mode).

Both acceptance modes define exactly the same class of languages — the Context-Free Languages — and there are standard constructions to convert a PDA using one mode into an equivalent PDA using the other.

Question 7

Define the language accepted by a PDA.

Answer:

The language accepted by a PDA M , denoted $L(M)$, is the set of all input strings w such that, starting from the initial ID (q_0, w, Z_0) , the PDA can reach some ID satisfying its acceptance condition — either (q, ϵ, γ) with $q \in F$ for final-state acceptance, or (q, ϵ, ϵ) for empty-stack acceptance.

In other words, $L(M) = \{ w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (\text{accepting configuration}) \}$. The class of all languages that can be accepted by some PDA is exactly the class of Context-Free Languages.

Question 8

How does a PDA process a string using IDs?

Answer:

Processing begins at the initial ID (q_0, w, Z_0) , where w is the full input string and Z_0 is the initial stack symbol. At each step, the PDA applies a transition from δ based on the current state,

the next input symbol (or ϵ), and the symbol on top of the stack; this produces a new ID with the input pointer advanced (if a symbol was consumed), a new current state, and an updated stack.

This is written as a sequence of moves $(q_0, w, Z_0) \vdash (q_1, w_1, \gamma_1) \vdash (q_2, w_2, \gamma_2) \vdash \dots$ until the input is fully consumed. The string is accepted if the final ID satisfies the chosen acceptance condition (final state or empty stack).

Question 9

Differentiate between Deterministic PDA and Non-Deterministic PDA.

Answer:

- Deterministic PDA (DPDA): for every combination of state, input symbol (or ϵ), and stack top, there is at most one applicable transition, and if an ϵ -move is possible from a configuration, no other move on an actual input symbol is allowed from that same configuration. A DPDA has exactly one computation path for any input.
- Non-Deterministic PDA (NPDA): multiple transitions can be applicable for the same state, input symbol, and stack-top combination, so the machine can explore several possible computation paths (conceptually in parallel) and accepts if at least one path leads to an accepting configuration.
- Power difference: NPDAs recognize exactly the Context-Free Languages, whereas DPDAs recognize only a strict subset called Deterministic Context-Free Languages (DCFLs). This means some context-free languages (e.g., certain languages requiring lookahead/ambiguity resolution) cannot be recognized by any DPDA, even though an NPDA exists for them.

Question 10

Why are PDAs equivalent to Context-Free Grammars (CFG)?

Answer:

PDAs and CFGs are equivalent in expressive power: for every CFG there exists a PDA that accepts exactly the language generated by that grammar, and for every PDA there exists a CFG that generates exactly the language it accepts. This is proven using two standard constructions:

- CFG $\rightarrow$ PDA: build a PDA that simulates leftmost derivations by pushing grammar symbols onto the stack and replacing a nonterminal on top of the stack with the right-hand side of one of its productions, matching terminals directly against input.
- PDA $\rightarrow$ CFG: build a grammar whose nonterminals represent 'the PDA can move from state p with empty-stack-relative-symbol X to state q ', with productions derived from the

PDA's transitions, so that derivations in the grammar correspond exactly to accepting computations of the PDA.

Because each direction preserves the language exactly, PDAs and CFGs define the same class of languages — the Context-Free Languages.

Question 11

What types of languages are accepted by PDA? Give examples.

Answer:

PDAs accept exactly the Context-Free Languages (CFLs) — languages that can be generated by some Context-Free Grammar. These typically involve one nested or matching structure that requires unbounded but single-comparison memory.

- $L = \{a^n b^n \mid n \geq 0\}$ — equal numbers of a's followed by b's.
- $L = \{ww^R \mid w \in (a+b)^*\}$ — even-length palindromes (w followed by its reverse).
- Balanced parentheses / well-nested bracket expressions, e.g., matching '(', ')' pairs.
- Arithmetic expression languages generated by grammars such as $E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid \text{id}$.

Question 12

What is a Turing Machine? How is it more powerful than PDA?

Answer:

A Turing Machine (TM) is a theoretical computational model consisting of an infinite tape (used as both input and unbounded read/write memory), a tape head that can move left or right and read/write symbols, a finite set of states, and a transition function. It is the most powerful standard model of computation and is capable of expressing any algorithm (Church–Turing thesis).

A TM is more powerful than a PDA because its tape allows unrestricted random-access reading and rewriting at any position, any number of times, whereas a PDA's stack only allows last-in-first-out access. This lets a TM solve problems requiring multiple independent comparisons or arbitrary revisiting of data (e.g., $a^n b^n c^n$, or checking if $w = ww$), which are provably beyond the power of any PDA. TMs recognize the Recursively Enumerable Languages, a strictly larger class than the Context-Free Languages recognized by PDAs.

Question 13

Define the components of a Turing Machine.

Answer:

A Turing Machine is formally defined as a 7-tuple: $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:

- Q — a finite set of states.
- Σ — the input alphabet (symbols allowed in the input, $\Sigma \subseteq \Gamma$).
- Γ — the tape alphabet (includes Σ and the blank symbol B).
- δ — the transition function: $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ (state/symbol pair maps to new state, symbol to write, and head direction).
- q_0 — the initial state, $q_0 \in Q$.
- B — the blank symbol, used to fill unused tape cells.
- F — the set of final/accepting states, $F \subseteq Q$.

Question 14

Explain tape, head, and state transitions in a Turing Machine.

Answer:

- Tape: an infinite (in at least one direction) sequence of cells, each capable of holding a single symbol from the tape alphabet Γ ; it serves as both the input and the machine's read/write working memory.
- Head: a read/write head positioned over one tape cell at a time; on each step it reads the symbol under it, and based on the transition function, writes a (possibly new) symbol in that cell and then moves one cell Left or Right (or stays, in some variants).
- State transitions: governed by $\delta(\text{current state, symbol read}) = (\text{next state, symbol to write, direction to move})$. This single rule captures the entire behavior of the machine at each step, and repeated application drives the computation until a halting/accepting state is reached (or the machine runs forever/rejects).

Question 15

Explain programming techniques for Turing Machines.

Answer:

Because writing a raw Turing Machine transition table directly can be tedious, several standard 'programming techniques' are used to design TMs in a structured, modular way:

- Storage in finite control: using the machine's states themselves to remember a small, finite amount of extra information (e.g., the last symbol read) instead of writing it to tape.

- Multiple tracks: dividing each tape cell into several parallel sub-cells (an enlarged alphabet $\Sigma \times \Sigma \times \dots$) to conveniently store related data side by side, e.g., an original symbol and a marker in the same cell.
- Checking off symbols: marking (overwriting) symbols on the tape as they are matched/processed, which is useful for comparing two blocks of symbols (e.g., verifying equal counts) without extra memory.
- Shifting over: temporarily moving a block of tape symbols left or right to make room to insert new symbols.
- Subroutines: designing reusable, self-contained blocks of states with defined entry and 'return' states, which larger machines can invoke much like functions/procedures in ordinary programming, promoting modular TM design.

Question 16

What is storage in finite control in Turing Machines?

Answer:

Storage in finite control is a technique where a Turing Machine uses its own states to remember a small, fixed amount of extra information, rather than writing that information onto the tape. Effectively, each state is treated as a pair (control state, remembered symbol/value), so the total number of states grows to accommodate all the combinations of 'phase of computation' and 'small piece of remembered data'.

This is useful for tasks like remembering the most recently read symbol so it can be compared against the next symbol read, without needing to move the head back or use extra tape space — since only a finite amount of information needs to be remembered, encoding it into the states keeps the design simple and tape usage minimal.

Question 17

Compare FA, PDA, and TM based on memory, power, and language acceptance.

Answer:

- Memory — FA: none beyond its finite states, cannot count. PDA: one stack (LIFO), unbounded but restricted access. TM: infinite tape with unrestricted random-access read/write.
- Power — FA is the weakest, recognizing only Regular Languages; PDA is strictly more powerful, recognizing Context-Free Languages; TM is the most powerful, recognizing Recursively Enumerable Languages and capable of expressing any algorithm.

- Language acceptance — FA accepts Regular Languages (e.g., strings matching a pattern with no unbounded counting); PDA accepts Context-Free Languages (e.g., $a^n b^n$, balanced parentheses, one nested/matching structure); TM accepts Recursively Enumerable Languages (e.g., $a^n b^n c^n$, general algorithmic problems, anything computable).
- Hierarchy — the classes form a strict chain: Regular $\subset$ Context-Free $\subset$ Recursively Enumerable, corresponding to FA $\subset$ PDA $\subset$ TM in computational power.

Question 18

What is DFA?

Answer:

A Deterministic Finite Automaton (DFA) is a finite automaton in which, for every state and every input symbol, there is exactly one defined transition to a next state (no ambiguity and no ϵ -moves). It is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where $\delta: Q \times \Sigma \rightarrow Q$ is a total function.

A DFA processes an input string symbol by symbol, following exactly one path of states, and accepts the string if it ends in a final state after the entire input is consumed. DFAs recognize exactly the Regular Languages, the same class recognized by Non-Deterministic Finite Automata (NFA), since every NFA can be converted into an equivalent DFA (subset construction), though the DFA may need more states.

Question 19

Which language is accepted by a Turing Machine?

Answer:

A Turing Machine accepts exactly the class of Recursively Enumerable (RE) languages, also called Type 0 languages in the Chomsky hierarchy. A language L is recursively enumerable if there exists a TM that halts and accepts every string in L ; for strings not in L , the TM may either halt and reject or run forever (never halting).

If, in addition, the TM always halts on every input (whether accepting or rejecting), the language is called Recursive (or Decidable) — a subclass of the Recursively Enumerable languages. This makes the TM the most general language-acceptor among the standard automata models, capable of recognizing every language that is algorithmically describable.

Question 20

How are Turing Machines applied in real-world applications?

Answer:

Although no physical machine is literally a Turing Machine (since it requires infinite tape/memory), the Turing Machine model serves as the theoretical foundation for understanding what is and is not computable, and its principles underlie many real-world systems and ideas:

- Modern computers and programming languages: general-purpose computers are physical approximations of Turing Machines (Turing-complete), and the model justifies why any algorithm expressible in one Turing-complete language can, in principle, be expressed in another.
- Compiler design and formal language theory: TM-based reasoning underlies the study of decidability/undecidability of problems relevant to compilers, such as whether certain code-analysis problems can be solved algorithmically at all.
- Algorithm analysis and computability theory: the TM model is used to formally define what 'computable' means, to prove certain problems are undecidable (e.g., the Halting Problem), and to study computational complexity classes (P, NP, etc.), which underpin cryptography and algorithm design.
- Automata-based tools: TM-inspired ideas appear in the design of parsers, text processing tools, and simulators used in software engineering and formal verification.